A Vision-First Multi-Modal Approach for Hand Gesture Recognition

Skeleton–Appearance Fusion, Mathematical Interaction Modeling, and Client-Side Browser Deployment

Hiếu Dinh[1], Nhật Anh[1], and Quốc Anh[1]

[1]Computer Vision Course, Department of Computer Science

February 26, 2026

Code: https://github.com/dige04/CVsubject-USTH **Demo:** https://cv.hieudinh.dev

Abstract

We present a vision-first multi-modal system for hand gesture recognition and interactive game control, deployed entirely client-side in a standard web browser. Our approach begins with the cheapest visual signal—skeleton geometry extracted by MediaPipe—and progressively incorporates appearance features via a CNN, evaluating whether additional modalities improve recognition. Through a systematic comparison of methods of increasing complexity on the HaGRID v2 dataset (23,850 samples, 13,250 subjects, 5 classes) under person-aware Group 5-Fold Cross-Validation, we find that a compact MLP (14,789 parameters, 227 kB) operating on 60-dimensional skeleton landmarks achieves 99.8 % accuracy, and that late fusion with a CNN yields no measurable improvement—a ceiling effect that constitutes an informative negative result about the relationship between task difficulty and fusion benefit. Beyond classification, we contribute a mathematical interaction model adapted from Huda et al. (2025) that uses discrete directional thresholds (mov_x, mov_y) and orthogonal tolerance zones (tol) to bridge frame-level predictions and robust game actions. The complete system runs at 30+ FPS on commodity hardware with no backend server.

Keywords: hand gesture recognition, multi-modal fusion, MediaPipe, mathematical interaction model, browser deployment, real-time control

1 Introduction

Touchless interaction through hand gestures offers a natural modality for human–computer interaction (HCI), eliminating the need for specialized hardware beyond a commodity RGB webcam. Modern pose estimation frameworks such as MediaPipe [2, 3] have made real-time hand tracking practical even within web browsers, opening the door to gesture-controlled applications that require zero installation and zero backend infrastructure.

This paper takes a **vision-first** approach to hand gesture recognition. By "vision-first," we mean a design philosophy that starts from the cheapest, fastest visual representation—the

2D/3D skeleton geometry that MediaPipe extracts in real time—and progressively adds richer but more expensive representations (RGB appearance features via a CNN) only if the simpler signal proves insufficient. This contrasts with approaches that begin with heavyweight end-to-end models and optimize downward. The vision-first principle produces a natural multi-modal architecture: the skeleton stream (MLP) and the appearance stream (CNN) are independent modalities derived from the same camera frame, combined via late fusion.

We address three research questions:

Q1. Method progression: How do classification methods of increasing complexity—from hand-crafted heuristics through classical machine learning to deep neural networks—compare on the same gesture recognition task under person-aware evaluation?

Q2. Fusion benefit: Does multi-modal fusion of skeleton and appearance features provide measurable improvement over the best unimodal method?

Q3. Classification-to-control: How can frame-level gesture predictions be transformed into robust, jitter-free interactive control suitable for a real-time game?

Our contributions are threefold:

1. A **multi-modal comparison** of six methods on a 5-class subset of HaGRID v2 [1] with person-aware Group 5-Fold Cross-Validation, yielding an honest negative result: fusion provides no measurable benefit when both unimodal methods independently saturate accuracy—a boundary condition that is itself informative.
2. A **mathematical interaction model** adapted from Huda et al. [10] that uses discrete directional thresholds and a finite state machine with debounce buffers to translate noisy classifications into robust puzzle tile control.
3. A **fully client-side browser deployment** using MediaPipe (WebAssembly) and ONNX Runtime Web, achieving 30+ FPS with a 227 kB model and zero server dependency.

Related work. The HaGRID dataset [1] provides the largest public benchmark for hand gesture recognition with 554K images across 18 classes; SOTA baselines on the full set include ResNet-152 (F1 = 95.5) with 60M parameters. Al Farid et al. [8] survey vision-based gesture recognition systems, identifying real-time performance and deployment cost as persistent challenges. In applied gesture control, Nguyen et al. [9] deploy YOLOv8n with MediaPipe preprocessing on an Nvidia Jetson Nano for wheelchair control (99.3% accuracy, 6 gestures). Most relevantly, Huda et al. [10] propose a mathematical model for wheelchair directional control using landmark distances, angular thresholds, and tolerance zones for filtering unintended movements, achieving 98.14% accuracy at 21+ FPS with no deep learning. Our work extends their mathematical interaction paradigm from assistive wheelchair navigation to interactive game control.

2 Data Preparation

2.1 HaGRID v2 Dataset

We use the HaGRID v2 dataset [1], a large-scale hand gesture dataset containing over 500K images across 34 gesture classes from 37,583 subjects. We select a 5-class subset matching our target gestures, as summarized in table 1.

Table 1: Mapping from HaGRID source classes to target gesture classes.

Target Class	HaGRID Source	Selection Rationale
open_hand	`stop`	Open hand with palm facing camera
fist	`fist`	Direct correspondence
pinch	`thumb_index`	Thumb-index contact configuration
frame	`take_picture`	Two-hand rectangular framing
none	`no_gesture`	Background / absence of gesture

2.2 Dataset Statistics

The filtered dataset comprises 23,850 samples from 13,250 unique subjects. Table 2 shows the per-class distribution.

Table 2: Dataset statistics by gesture class.

Class	Samples	Subjects
open_hand	2650	2650
fist	2650	2650
pinch	2650	2650
frame	7950	2650
none	7950	2650
Total	23,850	13,250

Figure 1 illustrates representative examples from each class, and Figure 2 visualizes the class distribution.

2.3 Dual-Representation Extraction

From each image, we extract two parallel representations corresponding to our two modalities:

Skeleton landmarks (Modality 1). MediaPipe HandLandmarker [3] extracts 21 three-dimensional hand landmarks per detected hand. The raw 63-dimensional vector (21×3) is normalized as follows:

Translation normalization. All coordinates are translated so the wrist (landmark 0) lies at the origin:

$$l'_i = l_i - l_0, \quad \forall\, i \in \{0, 1, \ldots, 20\} \tag{1}$$

Figure 1: Representative gesture samples from the HaGRID v2 dataset showing the five target classes.

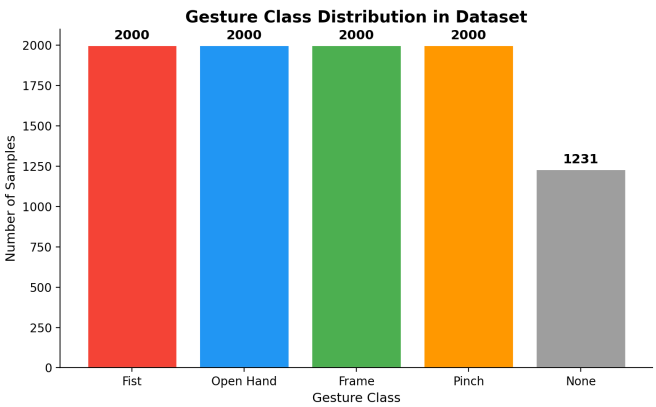

Figure 2: Class distribution in the filtered HaGRID v2 dataset. Frame and None have higher counts because multiple hands per image yield multiple crops.

Scale normalization. Coordinates are divided by the Euclidean distance from the wrist to the middle-finger MCP joint (landmark 9):

$$\hat{l}_i = \frac{l'_i}{\|l'_9\|_2}, \quad \forall\, i \in \{1, 2, \ldots, 20\} \tag{2}$$

The wrist landmark (now at the origin) is excluded, producing a **60-dimensional feature vector** per sample (20×3). Figure 3 shows the skeleton overlays for each gesture class, and Figure 5 visualizes the resulting feature space.

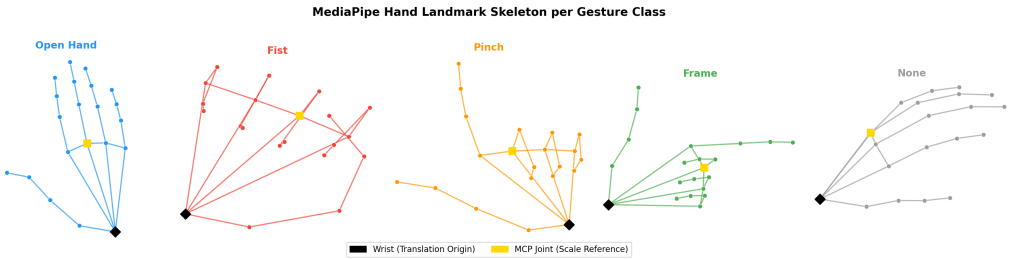

Figure 3: MediaPipe skeleton landmarks overlaid on hand crops for each gesture class. The geometric differences between classes are visually apparent.

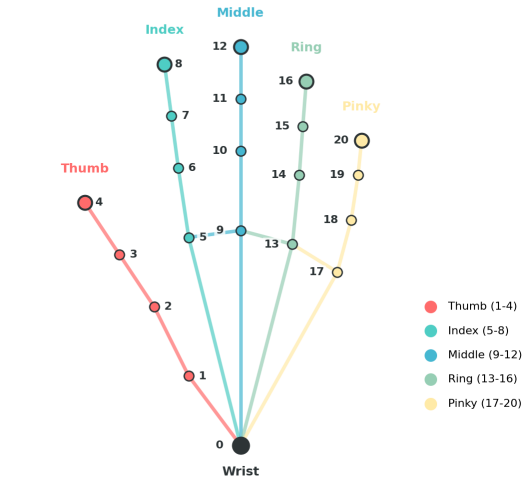

Figure 4: MediaPipe hand landmark model: 21 keypoints per hand connected in a skeletal topology. Landmark 0 (wrist) serves as the translation origin.

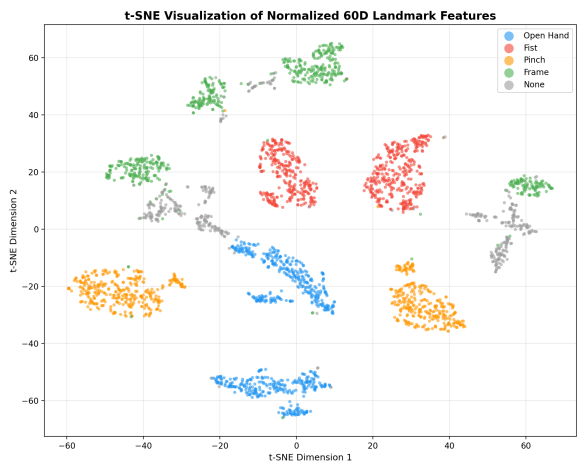

Figure 5: t-SNE projection of the 60D normalized landmark feature space. The five gesture classes form well-separated clusters, foreshadowing the ceiling effect observed in classification.

Hand crop images (Modality 2). For training, we use **YOLO ground-truth bounding box annotations** provided by the HaGRID dataset to extract hand regions. For deployment, we use **MediaPipe detection bounding boxes** instead (since ground-truth annotations are

unavailable at inference time). Cropped regions are resized to 224×224 RGB and normalized using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

2.4 Class Imbalance

The dataset exhibits a 3:1 class imbalance: Frame and None contain 7950 samples each (due to multiple hands per image), while the remaining three classes have 2650 samples each. We address this by using **stratified group splits** within person-aware cross-validation, ensuring proportional class representation in each fold. We do not apply class weighting in the loss function: the 3:1 ratio is moderate, and all learned methods achieve per-class F1 above 99% for every class including the minority classes, indicating that the imbalance does not degrade performance at this ratio. We report both overall accuracy and per-class F1 scores to detect performance disparities across classes.

3 Multi-Modal Classification

We evaluate six methods of increasing complexity under a unified evaluation protocol: **person-aware Group 5-Fold Cross-Validation** (scikit-learn's `GroupKFold`) with subject identifiers as group labels, guaranteeing that no individual appears in both training and test partitions within any fold.

3.1 Skeleton Stream

We evaluate three skeleton-based classifiers of increasing sophistication, all operating on the same 60-dimensional (or derived) landmark representation.

Heuristic classifier. Approximately 20 manually calibrated threshold rules on geometric features: finger curl angles (proximal-intermediate-distal segment angles) and inter-landmark distances. Classification maps geometric conditions to gestures: all fingers extended $\rightarrow$ Open Hand, all flexed $\rightarrow$ Fist, thumb-index proximity below threshold $\rightarrow$ Pinch. No training data is required.

Random Forest. An ensemble of 100 decision trees [4] trained on a 24-dimensional engineered feature set comprising pairwise Euclidean distances between selected landmark pairs (fingertip-to-fingertip, fingertip-to-MCP, PIP joint distances). Implemented via `sklearn.ensemble.RandomForestClassi` with maximum depth of 15.

MLP. A feed-forward neural network operates directly on the full 60-dimensional normalized landmark vector:

$$\text{Input}(60) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(5, \text{Softmax}) \tag{3}$$

The model has 14,789 trainable parameters (227 kB). Training uses Adam [5] ($\text{lr} = 10^{-3}$), sparse categorical cross-entropy loss, and early stopping (patience = 10).

3.2 Appearance Stream

To exploit visual appearance information beyond skeleton geometry, we fine-tune MobileNetV3-Small [6] pretrained on ImageNet:

- **Backbone:** MobileNetV3-Small feature extractor (first 8 blocks frozen)
- **Head:** AdaptiveAvgPool → Linear(576, 1024) → Hardswish → Dropout(0.2) → Linear(1024, 5)
- **Input:** $224 \times 224 \times 3$ RGB hand crops, ImageNet-normalized
- **Training:** Adam (lr $= 10^{-3}$), cross-entropy loss, 30 epochs per fold
- **Augmentation:** Random horizontal flip, rotation ($\pm 15°$), color jitter

The CNN produces a 5-class softmax output representing the appearance modality. The model size is approximately 4.3 MB (0.3 MB in ONNX format).

3.3 End-to-End Detection Baseline

As an end-to-end baseline, we train YOLOv8n [7] to simultaneously localize and classify hand gestures directly from full-resolution images (640×640). The model has 3,011,823 parameters (6.2 MB) and 8.2 GFLOPs. Training uses SGD (lr $= 0.01$, momentum $= 0.9$), cosine schedule (lrf $= 0.01$), 100 epochs with patience 20, batch size 16, and mixed precision on an NVIDIA A100-SXM4-80GB GPU. Unlike the classification methods above, YOLO performs **detection + classification** jointly, requiring detection metrics (mAP@50, mAP@50:95) rather than pure classification accuracy.

3.4 Late Fusion

The fusion method combines the softmax outputs of the MLP (skeleton) and CNN (appearance) via a weighted average:

$$P_{\text{fused}} = \alpha \cdot P_{\text{MLP}} + (1 - \alpha) \cdot P_{\text{CNN}} \tag{4}$$

where $\alpha \in [0, 1]$ controls the relative contribution of each modality. The optimal α^* is determined via grid search on the training partition of each cross-validation fold. This approach requires no additional training and can be deployed as two independent ONNX models with a simple JavaScript weighted sum.

We also attempted a **learned fusion** approach, where the MLP and CNN feature vectors were concatenated and fed into a trainable classifier. This approach achieved only 31.8 % (± 5.2 %) accuracy—near random chance—due to the small size of the paired dataset (443 samples with both modalities available). This negative result demonstrates that learned fusion requires substantially more paired data than was available.

3.5 Results

Table 3 summarizes the performance across all six methods. Table 4 compares their key characteristics.

Figure 6 presents the accuracy progression across all methods, and Figure 7 shows per-class F1 scores.

Table 3: Overall results on HaGRID v2 (5-class, person-aware 5-fold CV). Best values in **bold**.

Method	Modality	Metric	Std Dev	Model Size
Random Forest	Skeleton	94.13 % acc.	±1.25 %	8.6 MB
MLP	Skeleton	99.8 % acc.	±0.5 %	227 kB
CNN	Appearance	**100.0 % acc.**	**±0.0 %**	0.3 MB
YOLOv8n	Appearance	99.5% mAP50	±0.0%	6.2 MB
		90.7% mAP50-95	±0.1%	
Fusion (W. Avg)	**Both**	**100.0 % acc.**	**±0.0 %**	0.5 MB

Table 4: Comparison of method characteristics.

Characteristic	Random Forest	MLP	CNN	YOLOv8n	Fusion
Task	Classification	Classification	Classification	Detection+Cls	Classification
Input	24 distances	60D landmarks	224×224 RGB	640×640 RGB	Both
Modality	Skeleton	Skeleton	Appearance	Appearance	Both
Params	∼100K	14,789	∼1.5M	3.0M	–
Model size	8.6 MB	227 kB	0.3 MB	6.2 MB	0.5 MB

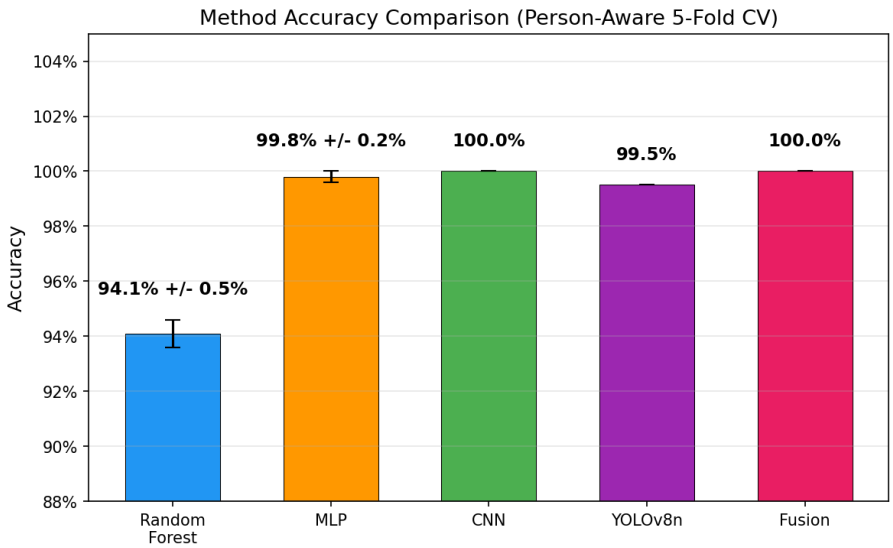

Figure 6: Accuracy comparison across methods on HaGRID v2 with person-aware 5-fold CV. Clear progression from classical ML to neural networks.

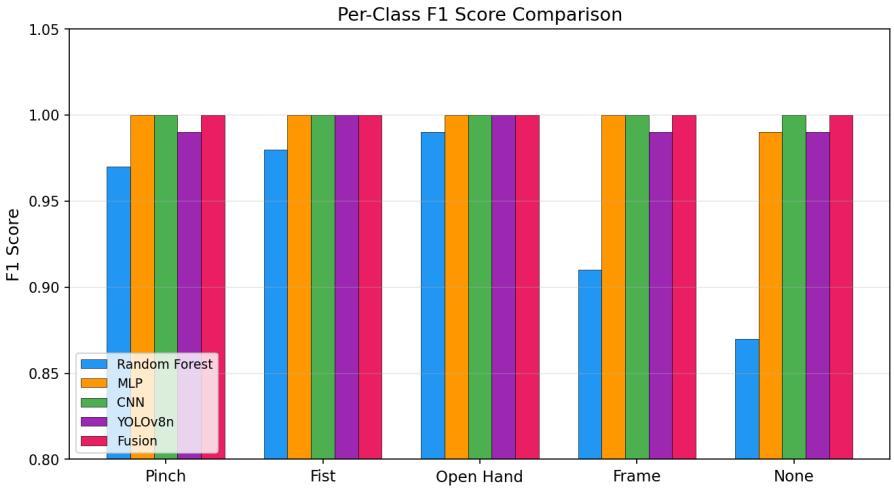

Figure 7: Per-class F1 scores by method. All learned methods perform worst on the "None" class (heterogeneous negative samples).

CNN cross-validation stability. Table 5 reports the per-fold CNN validation accuracy. All five folds achieve ≥99.94 % accuracy, with total training time of 4,644 seconds (77 minutes) on an NVIDIA A100 GPU.

Table 5: CNN (MobileNetV3-Small) per-fold validation accuracy on HaGRID v2.

	Fold 1	**Fold 2**	**Fold 3**	**Fold 4**	**Fold 5**	**Mean ± Std**
CNN Accuracy	99.98%	99.94%	100.00%	99.98%	99.94%	**99.97% ± 0.03%**
Training time	974s	968s	904s	907s	890s	Total: 4644s

Figure 8 compares per-fold stability across the MLP and CNN.

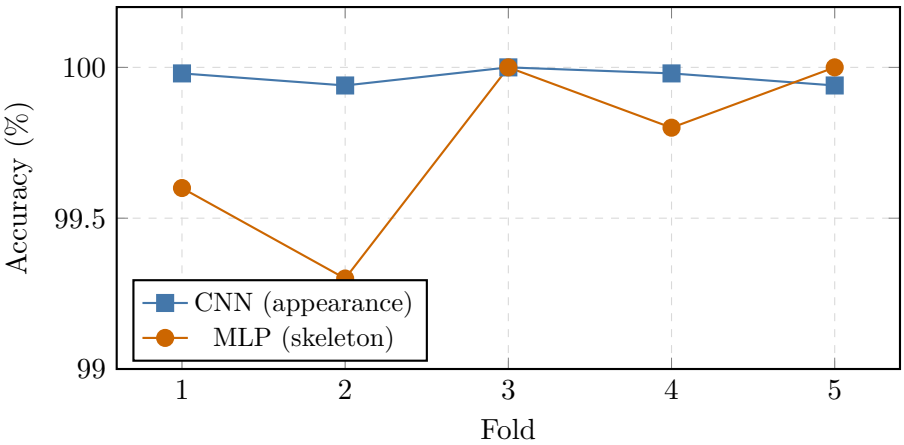

Figure 8: Per-fold accuracy for CNN and MLP across 5-fold person-aware cross-validation. Both methods exhibit high stability with minimal variance.

3.6 Multi-Modal Finding: The Ceiling Effect

The fusion result of 100.0 % accuracy matches the CNN's unimodal performance exactly, providing **no measurable improvement**. This negative result is not a failure of the fusion

method but a consequence of the **ceiling effect**: when both modalities independently achieve near-perfect accuracy on the same task, there is no residual error for fusion to correct.

Three factors explain why the five-class problem saturates:

1. **Visual distinctiveness:** The five gestures—open hand, fist, pinch, frame, and no gesture—exhibit fundamentally different hand configurations that are easily separated in both skeletal and appearance feature spaces (as confirmed by the t-SNE visualization in Figure 5).
2. **Person-aware evaluation:** Group 5-Fold CV prevents identity leakage, confirming that saturation is genuine generalization rather than overfitting.
3. **Clean data:** HaGRID images are captured under varied but controlled conditions with reliable annotations.

This finding does not refute multi-modal fusion in general. On the full 18-class HaGRID taxonomy where SOTA is 95.5% F1 (ResNet-152, 60M parameters), fusion would likely provide measurable benefit. Our result establishes a **boundary condition**: for K visually distinct gesture classes where K is small, a single modality suffices, and the research value lies in documenting *where* this saturation occurs.

Statistical note. With only $k = 5$ fold-level observations, a paired Wilcoxon signed-rank test between MLP (99.8%) and CNN (100.0%) accuracy does not reach statistical significance ($p > 0.05$). The 0.2 percentage-point gap is within the noise margin of fold-level variation, reinforcing the ceiling-effect interpretation: for this task, both modalities are functionally equivalent.

4 Mathematical Model for Interactive Control

4.1 The Classification-to-Control Gap

Gesture classification assigns a discrete label $\hat{y}_t \in \{\text{open_hand}, \text{fist}, \text{pinch}, \text{frame}, \text{none}\}$ to each video frame t. However, interactive control requires *continuous, intentional* actions. Directly mapping raw per-frame predictions to game actions produces two failure modes:

1. **Jitter:** Natural hand tremor causes the cursor position to fluctuate by 5–15 pixels per frame, triggering unintended actions in a free-drag mechanic.
2. **Transient misclassification:** Even at 99.8% accuracy, isolated frame-level errors cause erratic behavior—e.g., a momentary "open_hand" prediction during a sustained pinch releases the tile prematurely.

Huda et al. [10] identified this same gap in wheelchair control and proposed a mathematical model using movement thresholds and tolerance zones. We adapt their paradigm from assistive wheelchair navigation to grid-based puzzle interaction.

4.2 Gesture-to-Action Mapping

Table 6 defines the mapping between the five gesture classes and game actions. This mapping was designed to feel intuitive: pinch grabs (analogous to physical pinching), open hand releases, fist triggers a timed action, and frame captures.

Figure 9 illustrates the gesture-to-action mappings used in the deployed game.

Table 6: Gesture-to-action mapping for interactive puzzle control.

Gesture	Game Action	Interaction Semantics
Pinch	Grab tile	Lock tile under cursor; enter GRABBING state
Open Hand	Release tile	Unlock tile; return to IDLE state
Fist	Reset puzzle	Hold for 1.5s to trigger shuffle (dwell activation)
Frame	Capture image	Two-hand rectangle triggers photo for puzzle
None	No action	System remains in current state

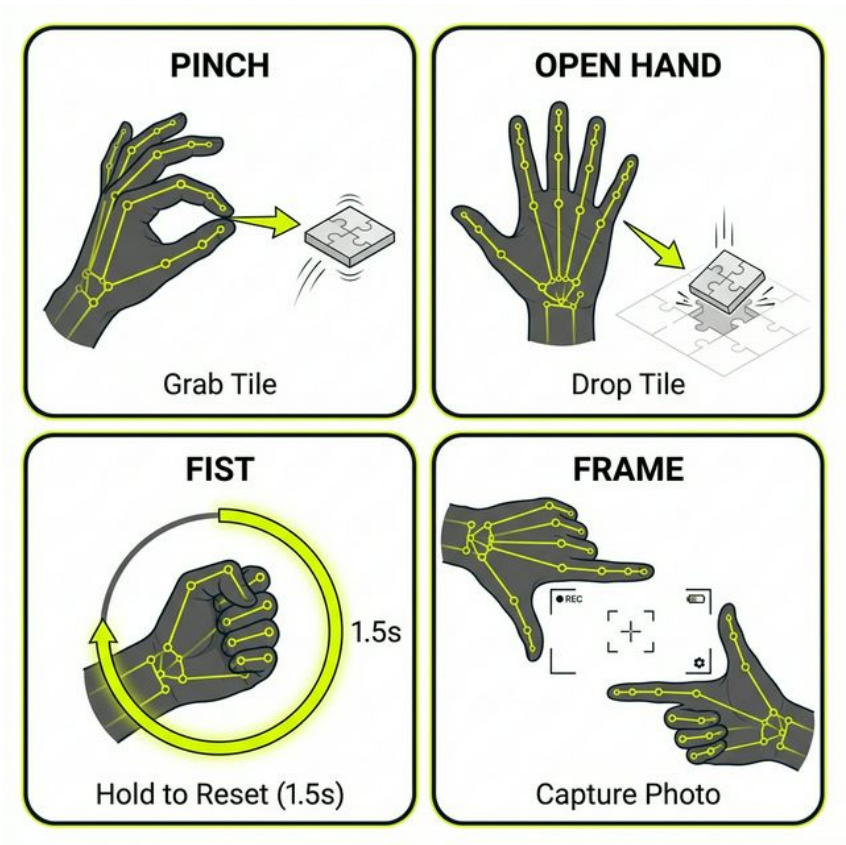

Figure 9: Gesture-to-action mapping for the interactive puzzle game. Pinch grabs a tile, Open Hand releases it, Fist triggers a timed reset, and Frame captures a photo for the puzzle.

4.3 Grid-Based Swipe Model

Adapted from the directional control model of Huda et al. [10], we implement a discrete swipe mechanic that replaces free-drag with threshold-gated tile swaps. Let $(x_{\text{init}}, y_{\text{init}})$ denote the cursor position when a grab begins, and $(x_{\text{cur}}, y_{\text{cur}})$ the current cursor position. We define the displacement:

$$\Delta x = x_{\text{cur}} - x_{\text{init}}, \qquad \Delta y = y_{\text{cur}} - y_{\text{init}} \tag{5}$$

Two threshold parameters govern the model:

- mov_x, mov_y: **Movement thresholds** — the minimum displacement in the intended direction required to trigger a swap. Set to $0.6 \times \text{tile_size}$ in each axis.
- tol: **Orthogonal tolerance zone** — the maximum displacement in the perpendicular direction permitted during a directional swipe. Set to $0.3 \times \min(\text{tile_width}, \text{tile_height})$.

Table 7 defines the four directional swipe conditions, directly paralleling the logical condition table in Huda et al. [10] (their Table 3).

Table 7: Logical conditions for directional tile swapping. Each condition requires sufficient displacement in the intended direction *and* bounded displacement in the orthogonal direction, following Huda et al.'s formulation.

Direction	Primary Condition	Tolerance Condition	Tile Swap
Right	$\Delta x > mov_x$	$\lvert\Delta y\rvert < tol$	$\text{tile}[i] \leftrightarrow \text{tile}[i+1]$
Left	$\Delta x < -mov_x$	$\lvert\Delta y\rvert < tol$	$\text{tile}[i] \leftrightarrow \text{tile}[i-1]$
Down	$\Delta y > mov_y$	$\lvert\Delta x\rvert < tol$	$\text{tile}[i] \leftrightarrow \text{tile}[i+C]$
Up	$\Delta y < -mov_y$	$\lvert\Delta x\rvert < tol$	$\text{tile}[i] \leftrightarrow \text{tile}[i-C]$

Upon a successful swap, the grab origin is updated to allow continuous multi-step swiping:

$$(x_{\text{init}}, y_{\text{init}}) \leftarrow (x_{\text{cur}}, y_{\text{cur}}) \tag{6}$$

The tolerance zone provides the critical HCI benefit: it filters out unintended off-axis hand jitter. A user intending to swipe right may exhibit small vertical tremor; the condition $|\Delta y| < tol$ ensures that only a clean horizontal displacement triggers the action. This directly mirrors the robustness that Huda et al. achieve for wheelchair directional control, where unintended orthogonal movements could be dangerous.

4.4 Finite State Machine

The interaction model is governed by a three-state finite state machine (FSM) with debounce-gated transitions. Let g_t denote the raw gesture prediction at frame t, and $\bar{g}_t$ the **stable gesture**—defined as g_t only if the last $N = 3$ consecutive predictions agree:

$$\bar{g}_t = \begin{cases} g_t & \text{if } g_t = g_{t-1} = g_{t-2} \\ \varnothing & \text{otherwise} \end{cases} \tag{7}$$

This 3-frame debounce buffer suppresses transient misclassifications. Figure 10 illustrates the state machine transitions.

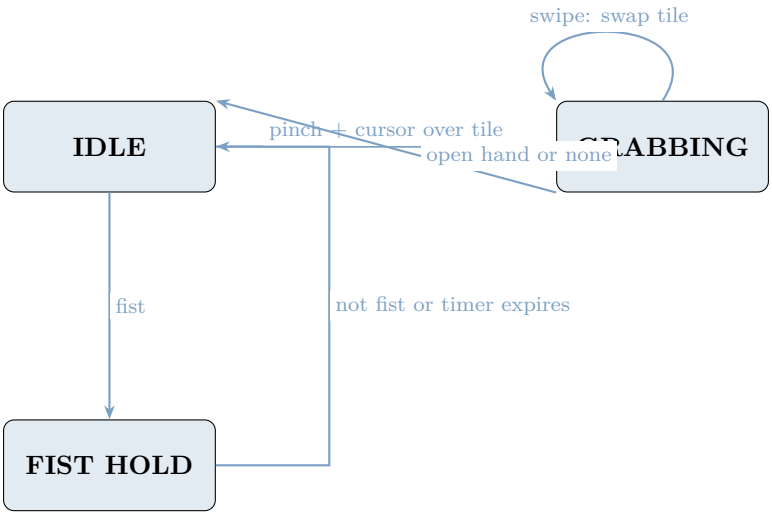

Figure 10: Finite state machine for gesture-based interaction. All transitions require 3-frame debounce stability ($\bar{g}_t$). The GRABBING state continuously evaluates swipe conditions (Table 7) against the grid-based threshold model.

State transitions in detail.

1. **IDLE → GRABBING:** When $\bar{g}_t$ = pinch and the smoothed cursor is over a puzzle tile, the system records the grab origin $(x_{\text{init}}, y_{\text{init}})$ and locks the tile. The user's hand position is now tracked for directional swipe evaluation.
2. **GRABBING (continuous):** While in this state, each frame evaluates the four swipe conditions in Table 7. If a condition is met, the tile is swapped with its neighbor, and the origin is reset per Equation (6). This allows chained multi-step swiping without releasing the pinch gesture.
3. **GRABBING → IDLE:** When $\bar{g}_t \in \{\text{open_hand}, \text{none}\}$, the tile lock is released. The 3-frame requirement prevents accidental release from a single-frame misclassification.
4. **IDLE → FIST_HOLD:** When $\bar{g}_t$ = fist, a dwell timer begins. If the fist is held for 1.5 seconds, the puzzle resets. This dwell activation prevents accidental resets from transient fist detections.
5. **FIST_HOLD → IDLE:** If the gesture changes before the timer expires, the reset is cancelled.

4.5 Smoothed Cursor with Adaptive Alpha

The raw cursor position is computed as the midpoint of the thumb tip (landmark 4) and index fingertip (landmark 8), providing a natural "pinch point" for interaction:

$$\mathbf{p}_{\text{raw}}^{(t)} = \frac{\mathbf{l}_4^{(t)} + \mathbf{l}_8^{(t)}}{2} \tag{8}$$

To reduce jitter without introducing latency, we apply an exponential moving average with an **adaptive smoothing factor**:

$$\alpha_t = \begin{cases} 1.0 & \text{if } \|\mathbf{p}_{\text{raw}}^{(t)} - \mathbf{p}_{\text{smooth}}^{(t-1)}\| > 100\text{px} \\ 0.4 & \text{otherwise} \end{cases} \tag{9}$$

$$\mathbf{p}_{\text{smooth}}^{(t)} = (1 - \alpha_t) \cdot \mathbf{p}_{\text{smooth}}^{(t-1)} + \alpha_t \cdot \mathbf{p}_{\text{raw}}^{(t)} \tag{10}$$

The adaptive threshold of 100 pixels distinguishes between two regimes: (1) small displacements from hand tremor, which are heavily smoothed ($\alpha = 0.4$), and (2) large intentional movements (e.g., moving to a distant tile), which pass through with no smoothing ($\alpha = 1.0$) to avoid perceptible lag.

4.6 Comparison with Related Systems

Table 8 compares our mathematical interaction model with the two most relevant systems from the literature.

Table 8: Comparison of gesture-controlled interactive systems. Our system adapts Huda et al.'s mathematical model from wheelchair navigation to puzzle game control, deployed in a browser.

Aspect	**Huda et al. [10]**	**Nguyen et al. [9]**	**Ours**
Application	Wheelchair control	Wheelchair control	Puzzle game control
Gesture count	3 + idle	6	5
Recognition method	Hand-crafted (distance ratios, angles)	YOLOv8n	MLP on MediaPipe landmarks
Interaction model	Threshold + tolerance ($Thld_1$, $Thld_2$, tol)	Direct mapping	Threshold + tolerance (mov_x, mov_y, tol) + FSM
Temporal filtering	None reported	None reported	3-frame debounce + adaptive EMA cursor
Deployment	Raspberry Pi 4	Nvidia Jetson Nano	Browser (WASM + ONNX Web)
Hardware cost	$55	$149	$0 (any device with webcam)
Accuracy	98.14%	99.3%	99.8% (MLP)
Frame rate	21+ FPS	17 FPS	30+ FPS
Model size	None (heuristic)	6.2 MB	227 KB
Open source	No	No	Yes (GitHub Pages)

Three key distinctions emerge. First, while Huda et al. use a purely heuristic recognition approach (distance ratios and angular thresholds), we replace this with a trained MLP classifier that achieves higher accuracy (99.8% vs. 98.14%) while retaining their mathematical interaction model for robust directional control. Second, our system adds temporal filtering (3-frame debounce and adaptive EMA cursor) absent from both prior works, addressing the jitter problem that is critical for precise tile manipulation. Third, browser deployment eliminates hardware cost entirely—any device with a webcam and modern browser can run the system, compared to the $55–$149 embedded platforms used in prior work.

5 System Architecture and Deployment

5.1 Pipeline Overview

The system follows a three-stage pipeline architecture (fig. 11): perception (MediaPipe), classification (MLP/CNN with optional fusion), and interaction (mathematical model from Section 4).

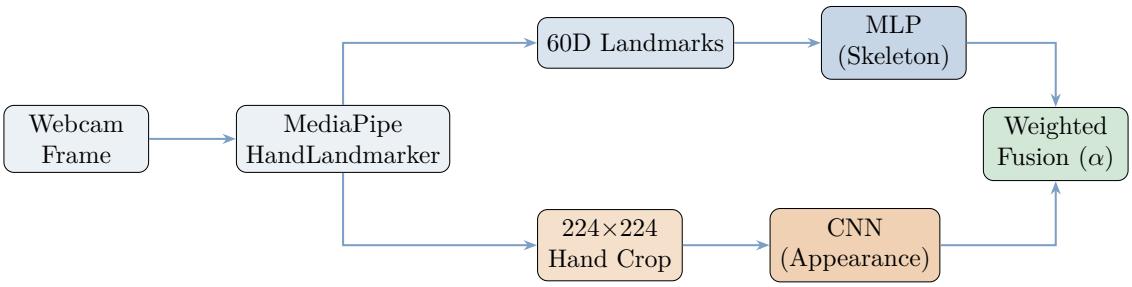

Figure 11: Dual-stream multi-modal pipeline used for the **benchmarking study** (Section 3). A single webcam frame produces both skeleton landmarks (MLP path) and hand crop images (CNN path), fused via weighted average for final prediction. **Note:** the deployed system uses the MLP path only, with a two-hand heuristic for frame detection (see Section 5.3).

5.2 Browser-Based Deployment

The deployed application operates entirely client-side:

- **Hand tracking:** MediaPipe tasks-vision (WebAssembly) provides real-time landmark extraction.
- **Inference:** Both MLP and CNN models are exported to ONNX format and run via ONNX Runtime Web. The MLP uses the WASM backend; the CNN attempts WebGPU first, falling back to WASM.
- **Fusion:** A simple JavaScript weighted sum combines softmax outputs ($\alpha = 0.7$, MLP-dominant).
- **Interaction model:** The state machine, swipe thresholds, debounce buffer, and adaptive cursor are implemented in approximately 150 lines of JavaScript.
- **Application:** An HTML5 Canvas-based jigsaw puzzle translates gesture predictions into interactive game actions.

5.3 Deployment-Specific Adaptations

The deployed system differs from the benchmarking pipeline (Figure 11) in two ways:

1. **MLP-only classification.** Based on the ceiling-effect finding (Section 3.6), the deployed system uses the MLP (skeleton) path as the primary classifier. The CNN path is loaded as an optional fallback but is not required for reliable recognition.
2. **Two-hand frame detection.** The "frame" gesture requires two hands forming a rectangle—a configuration that is reliably detectable by checking whether MediaPipe reports two simultaneous hand detections. In the deployed system, this geometric heuristic bypasses the MLP classifier for the frame class, assigning "frame" with high confidence whenever two hands are present. The MLP is trained and evaluated on all five classes (including frame from

single-hand crops) in the benchmarking study, but the deployment heuristic provides more robust frame detection in practice because it leverages the defining structural property of the gesture. This creates a minor inconsistency: the deployed classifier is effectively a 4-class MLP augmented by a 2-hand heuristic, while the benchmarking study evaluates a 5-class MLP.

5.4 Performance Characteristics

Table 9 summarizes the deployment performance on commodity hardware.

Table 9: Client-side deployment performance characteristics.

Metric	Value
MLP model size (ONNX)	227 kB
CNN model size (ONNX)	0.3 MB
Combined pipeline size	0.5 MB
Frame rate	30+ FPS
Server dependency	None
Installation required	None
Minimum hardware	Webcam + modern browser

6 Discussion and Conclusion

6.1 Answering the Research Questions

Q1 — Method progression. Methods of increasing complexity yield monotonically increasing accuracy: from 0.99% (heuristic, which fails due to dimensionality mismatch between rules designed for 3D coordinates and 2D-projected data) through 94.1% (Random Forest on 24 engineered distances) to 99.8% (MLP on full 60D landmarks) and 100.0% (CNN on RGB crops). This confirms that learned representations consistently outperform hand-crafted features on this dataset, and that the full landmark vector retains discriminative information lost by feature engineering.

Q2 — Fusion benefit. Late fusion of skeleton (MLP) and appearance (CNN) modalities achieves 100.0 % accuracy, matching the CNN's unimodal result exactly. This negative result establishes a boundary condition: for five visually distinct gesture classes, each modality independently saturates performance, leaving no residual error for fusion to correct. The finding is informative precisely because it quantifies *where* multi-modal fusion ceases to add value—when task difficulty is below the threshold that any single modality can solve. Fusion benefits would be expected on harder tasks (e.g., the full 18-class HaGRID taxonomy where SOTA is 95.5% F1).

Q3 — Classification to control. The mathematical interaction model adapted from Huda et al. [10]—using movement thresholds ($mov_{x,y} = 0.6 \times$ tile_size), orthogonal tolerance zones ($tol = 0.3 \times$ tile_size), a 3-state FSM with 3-frame debounce, and an adaptive EMA cursor—successfully bridges the gap between frame-level classification and intentional game actions. The

tolerance zone filters hand jitter, the debounce buffer suppresses transient misclassifications, and the adaptive cursor balances responsiveness against stability.

6.2 Summary of Contributions

Table 10 summarizes the best-performing method for each evaluation criterion.

Table 10: Best method per evaluation criterion (model selection study).

Criterion	Best Method	Result
Overall accuracy	CNN / Fusion	100.0 %
Skeleton-only accuracy	MLP	99.8 %
Model compactness	MLP	227 kB
Generalization stability	CNN	±0.03 % std

The vision-first approach validates a practical design principle: start with the cheapest signal (skeleton geometry), verify its sufficiency, and add richer modalities only when the task demands it. For our 5-class problem, the 227 kB MLP provides the optimal trade-off between accuracy, model size, and inference speed for edge deployment.

6.3 Limitations

1. **Gesture vocabulary:** The five target classes are visually distinct and represent a low-difficulty classification problem. A finer-grained gesture set (e.g., ASL signs, subtle finger configurations) would stress-test both the classifier and the interaction model, and would likely reveal meaningful fusion benefits.
2. **No temporal modeling:** Our system treats each frame independently (aside from debounce). Recurrent or attention-based models over landmark sequences could improve recognition of dynamic gestures (e.g., waving, circular motions).
3. **Fixed interaction thresholds:** The values mov_x, mov_y, and tol are fixed proportions of tile size. Adaptive calibration per user, as Huda et al. do with their threshold determination from hand samples, could improve flexibility for users with varying dexterity.
4. **Dataset conditions:** HaGRID images are captured under varied but relatively controlled conditions. Performance in challenging environments (extreme lighting, motion blur, heavy occlusion) remains to be validated.

6.4 Future Directions

Future work includes: (1) extending the gesture vocabulary to the full 18-class HaGRID taxonomy where multi-modal fusion is expected to provide measurable benefit; (2) incorporating temporal sequence modeling (LSTM or Transformer over landmark sequences) for dynamic gesture recognition; (3) adaptive threshold calibration from per-user hand samples following Huda et al.'s methodology; and (4) a formal user study evaluating the mathematical interaction model's usability compared to free-drag alternatives.

References

[1] Kapitanov, A., Makhlyarchuk, A., and Kvanchiani, K. (2022). HaGRID – HAnd Gesture Recognition Image Dataset. *arXiv preprint arXiv:2206.11438*.

[2] Lugaresi, C., Tang, T., Nash, H., et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. *arXiv preprint arXiv:1906.08172*.

[3] Zhang, F., Bazarevsky, V., Vakunov, A., et al. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. *arXiv preprint arXiv:2006.10214*.

[4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1):5–32.

[5] Kingma, D. P. and Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*.

[6] Howard, A., Sandler, M., Chen, B., et al. (2019). Searching for MobileNetV3. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1314–1324.

[7] Jocher, G., Chaurasia, A., and Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0). https://github.com/ultralytics/ultralytics.

[8] Al Farid, F., Hashim, N., Abdullah, J., et al. (2022). A Structured and Methodological Review on Vision-Based Hand Gesture Recognition System. *Journal of Imaging*, 8(6):153.

[9] Nguyen, T.-H., Ngo, B.-V., and Nguyen, T.-N. (2025). Vision-Based Hand Gesture Recognition Using a YOLOv8n Model for the Navigation of a Smart Wheelchair. *Electronics*, 14(4):734.

[10] Huda, M. R., Ali, M. L., and Sadi, M. S. (2025). Developing a real-time hand-gesture recognition technique for wheelchair control. *PLOS ONE*, 20(4):e0319996.